Ch 06: Structures

The C Programming Language (2^{ed})

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

EAST TEXAS A&M

Structures

Structure

A structure is a collection of one or more variables, possibly of different types, grouped together under a single name for convenient processing.

- Structures help to organize complicated data
  - Called "records", "object", "database entry or row", etc.
- A `struct` defines a new type
  - Sometimes called a user defined data type
  - Essentially adding a new type to the language you can declare variables of and use
  - Access members **structure member operator** "`.`"

To declare a structure (don't forget training `;`):

```
struct point {
  int x;
  int y;
};
struct point pt;
struct point maxpt = {320, 200 };
pt.x = 320;
pt.y = 200;
```

Pointers to Structures

- Can declare and use pointer to a struct, just like pointer to any defined type
- Special operator -> will access and dereference a pointer to a structure.

See `kr06/ex01.c`

```c
struct stuff {
  char *ptr;     // 8 bytes
  int num1;      // 4 bytes
  short num2;    // 2 bytes
  char byte;     // 1 bytes
  char bytes[8]; // 8 bytes
};

  struct stuff s;

  // a bunch of member access operations
  s.ptr = "hello";
  s.num1 = 0x11223344;
  s.num2 = 0x4142; // 2 bytes 0x41 = ascii A, 0x42 = ascii B
  s.byte = 'c';
  s.bytes[0] = 'h';

  // struct is allocated enough bytes to hold all members,
  // NOTE: the ptr and bytes are really pointers, so they are 8 bytes each
  // Question: why is size 24 here, is that what you expected?
  printf("sizeof(s): %ld\n\n", sizeof(s));

  printf("ptr: <%s>\n", s.ptr);
```

EAST TEXAS A&M

Summary

- point 1
- point 2

EAST TEXAS A&M

References